M08 - Files APIs Data Ingestion

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

This is the primary teacher.

Do not use the quick reference as the main lesson. Read the concept, run/inspect the demonstration, prove the result, then practise independently.

Contents

DE08-01 - Ingestion architecture: source -> landing -> staging

What you will be able to do

Explain and implement the boundary between a source system, immutable landing/raw copy and staging work area.

Why this matters in real Data Engineering

Ingestion is not merely reading a file. You need to preserve what arrived, know where transformation is allowed, and be able to replay without asking the source to recreate history.

Picture it this way

Think of a courier center: source is the sender, landing is the sealed receiving dock with timestamped parcels, staging is the workbench where parcels are unpacked and inspected.

Start from zero

- **Source** is where data originates: file drop, API, database or event stream.
- **Landing/raw** preserves what arrived with ingestion metadata; avoid business transformations here.
- **Staging** is a controlled work layer for parsing, typing, validation and normalization before downstream use.
- A good landing name includes identity such as source, logical date/run and content identity; the goal is replayability, not prettiness.
- Do not mutate a landed raw object after it is accepted. Write a new version/run instead.
- The same physical technology can hold multiple logical layers; the important boundary is the contract and allowed operations.

Teacher demonstration

```
# Stage 08 project folders
source/      # simulated external systems
landing/     # immutable received copies
staging/     # parsed/validated working output
quarantine/  # rejected objects/records
state/       # checkpoints / high-water marks
logs/        # run evidence
```

Read the example like English

- The folders are a teaching implementation of logical ingestion boundaries.
- Landing should answer: what exactly arrived and when?
- Staging may contain normalized shapes that are safe to rebuild from landing.
- Quarantine keeps bad input/evidence instead of making it disappear.

Expected check

boundary	allowed behavior
source	read from producer contract
landing	preserve raw + metadata
staging	parse/validate/normalize
quarantine	preserve rejected evidence

Common beginner traps

- Writing cleaned data back over the only raw copy.
- Using one folder named data/ for source, raw, outputs and failures so lineage becomes ambiguous.
- Calling staging a permanent truth layer without an explicit contract.

Now you do a similar task

Open PRACTICE_DATA/stage08_ingestion_project. Draw arrows from source files/API/database to landing, staging and quarantine. For each arrow state what may change and what must remain immutable.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

If staging can always be rebuilt, why is landing still valuable?

Answer in your own words before continuing.

DE08-02 - Defensive CSV ingestion

What you will be able to do

Land a CSV safely, inspect delimiter/header/schema and reject malformed records without corrupting the raw input.

Why this matters in real Data Engineering

CSV looks simple, so ingestion bugs are often silent: embedded delimiters, missing columns, inconsistent date/number formats and extra fields.

Picture it this way

A CSV is a form printed many times. Defensive ingestion first verifies the form layout before trusting values written in each row.

Start from zero

- Use a real CSV parser rather than split(",").
- Define required columns and expected types separately from the raw text representation.
- Count physical/business rows and record source filename/content hash before transformation.
- Parse values into a staging representation; conversion failures become rejection evidence, not guessed defaults.
- Separate file-level failures (wrong header/encoding) from row-level failures (bad amount).
- Reconcile accepted + rejected rows to the number of parsed source rows.

Teacher demonstration

```
python src/ingest_csv_TODO.py source/files/orders_2026-08-01.csv
# Then inspect landing/, staging/, quarantine/ and logs/
# Compare with source/files/orders_bad.csv only after the good run is understood.
```

Read the example like English

- The supplied TODO is your implementation workspace; reference solution remains closed until an attempt.
- A good file should create landed evidence and valid staging output.
- orders_bad.csv is designed to exercise schema/quality rejection paths.
- The final proof is row accounting + preserved raw object, not only "script exited 0".

Expected check

proof	what to save
raw identity	source name/hash/landing path
schema	required columns present
row accounting	accepted + rejected = parsed
source mutation	none

Common beginner traps

- fillna(0) or defaulting bad values during ingestion without business authority.
- Deleting rejected rows instead of recording reasons.
- Treating one successful sample file as proof the parser handles the contract.

Now you do a similar task

Implement the TODO CSV ingestion for the good file, then run the deliberately bad file. Save landing path, accepted/rejected counts and one rejection reason.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What is the difference between a file-level schema failure and a row-level data-quality failure?

Answer in your own words before continuing.

DE08-03 - JSON and nested-record ingestion

What you will be able to do

Inspect nested JSON shape, flatten only the fields needed downstream and preserve the original JSON evidence.

Why this matters in real Data Engineering

APIs often return nested arrays/objects. Blind normalization can duplicate parent fields or lose the relationship between parent and child records.

Picture it this way

A nested order is a folder containing an order cover sheet plus a stack of line-item slips. Flattening means deciding what one output row represents before copying fields.

Start from zero

- Inspect top-level type/keys and one complete sample before writing flattening code.
- Declare output grain first: one row per order, one row per order item, etc.
- When exploding a child array, parent identifiers are repeated intentionally to preserve the relationship.
- Missing optional nested fields need defaults/rejection rules; missing required identifiers usually cannot be silently ignored.
- Preserve raw JSON because flattening is lossy/restructuring.
- Reconcile parent count and child count separately so accidental fan-out is visible.

Teacher demonstration

```
python src/ingest_nested_json_TODO.py
# Input: source/json/orders_nested.json
# Goal: preserve raw JSON and produce a clearly-grained staging output.
```

Read the example like English

- The source includes nested orders/items.
- Your flattened table must state its row grain explicitly.
- If one order has three items, three line-grain rows are legitimate; that is not a duplicate by itself.
- Validation should distinguish legitimate repeated order_id from accidental duplicate line identities.

Expected check

question	proof
top-level shape	dict/list + keys
output grain	written sentence
parent count	reconciled
child rows	reconciled

Common beginner traps

- Using pandas.json_normalize before understanding nesting/grain.
- Calling repeated parent IDs duplicates at child grain.
- Discarding raw nested JSON after flattening.

Now you do a similar task

Inspect orders_nested.json manually and with Python. Write the intended flat grain, then implement the TODO and reconcile order count vs item-row count.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why can the same order_id legitimately appear several times after flattening nested order items?

Answer in your own words before continuing.

DE08-04 - Parquet ingestion and metadata inspection

What you will be able to do

Create/read Parquet and inspect schema, row groups and file metadata rather than treating it as a mysterious binary CSV.

Why this matters in real Data Engineering

Parquet stores typed columnar data and metadata that can make analytical reads more efficient and self-describing.

Picture it this way

CSV is like reading every field of every paper form line by line; Parquet is like a cabinet organized by columns with labels describing how blocks are stored.

Start from zero

- Parquet is columnar and stores schema/types with the file.
- A Parquet file is divided into row groups; within them column chunks/pages store encoded/compressed values.
- Projection means reading only selected columns; engines can also skip some data using metadata/statistics when predicates allow.
- File metadata inspection is a validation tool: schema, row count, row groups and compression.
- Parquet does not create ACID table semantics by itself; it is a file format.
- Choose file sizes/row groups for workload later; first learn to inspect what was actually written.

Teacher demonstration

```
python src/parquet_lab.py
# Read PARQUET_README_v1.0.txt
# Inspect schema, rows, row groups and a projected/filter read.
```

Read the example like English

- The supplied lab converts a controlled CSV to Parquet.
- Compare schema/row count before and after conversion.
- Notice the Parquet file is not meant to be read in a text editor.
- Metadata is part of the proof that the written file matches the contract.

Expected check

check	expected kind of evidence
schema	typed fields
row count	matches source business rows
projection	selected columns only
raw source	still preserved

Common beginner traps

- Calling Parquet a database/table transaction system.
- Converting to Parquet and never checking schema/row count.
- Assuming smaller file automatically means faster for every workload.

Now you do a similar task

Run the Parquet lab, record schema/row groups/row count, then read only two columns and a filtered subset. Explain what Parquet helped with and what it did not solve.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What information does a Parquet file preserve that a plain CSV normally does not encode strongly?

Answer in your own words before continuing.

DE08-05 - REST API contracts, status codes and timeouts

What you will be able to do

Interpret an API contract: method, endpoint, parameters, headers, success/error statuses, body shape and timeout.

Why this matters in real Data Engineering

An API can return valid HTTP responses that still represent errors, partial results or a changed contract. Reliable ingestion treats the contract explicitly.

Picture it this way

Calling an API is like submitting a form to a service desk: endpoint is the desk, method is the requested action, parameters/headers are form fields, status/body are the receipt and result.

Start from zero

- A GET request can still fail with 4xx/5xx statuses; do not parse every body as success data.
- Use explicit connect/read timeout behavior instead of allowing indefinite waits.
- Validate response Content-Type/body shape when the contract promises JSON.
- 4xx often means caller/request/auth problem; 5xx often means server-side/transient possibility, but policy depends on endpoint semantics.
- Record request correlation/run/page metadata without logging secrets.
- Contract validation includes required keys/types and pagination fields, not only HTTP status.

Teacher demonstration

```
# Terminal A
python mock_ingestion_api.py
# Terminal B
python src/api_contract_practice.py
# Inspect successful, invalid and slow/error endpoint behavior.
```

Read the example like English

- The mock API keeps behavior deterministic on your own PC.
- The practice script lets you observe status code + headers + JSON/body shape.
- Timeout is an intentional failure mode you should recognize.
- Do not change parsing code to "fix" an authentication or server error.

Expected check

layer	example evidence
transport	status/timeout
content	JSON contract keys
business	record validity after parsing

Common beginner traps

- Treating HTTP 200 as proof the complete dataset is valid.
- No timeout.
- Dumping Authorization headers/tokens into logs.

Now you do a similar task

Start the mock API, call at least one success and one error/slow route, and write which layer failed: transport/HTTP contract/content/business data.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

How can an HTTP 200 response still be unusable for your pipeline?

Answer in your own words before continuing.

DE08-06 - API authentication, pagination and rate limits

What you will be able to do

Fetch an authenticated paginated API completely while respecting rate-limit signals and hiding credentials.

Why this matters in real Data Engineering

Many real APIs deliberately return only one page and limit request rate. Missing pages silently creates incomplete data.

Picture it this way

A paginated API is a book handed to you page by page; authentication is your badge; rate limiting is the librarian telling you how fast you may request pages.

Start from zero

- Read credentials from environment/secret storage, not committed code.
- Pagination may use page numbers, offsets or opaque cursors; follow the contract-provided next signal.
- Track which pages/cursors completed so partial runs are visible.
- 429 means rate limit; honor Retry-After when supplied instead of hammering the service.
- Never infer completion just because the most recent page returned 200.
- Reconcile total API item count with landed/staged count and store run/page metadata.

Teacher demonstration

```
# Example shell (training value only)
export M08_API_TOKEN="training-token"
python src/ingest_api_TODO.py
# Inspect landing/API pages, state and logs. Do not print the token.
```

Read the example like English

- The TODO should retrieve all pages rather than one.
- Each raw response/page should be identifiable for replay/debugging.
- A correct rate-limit handler waits according to policy.
- Credential value must not appear in committed files/log evidence.

Expected check

proof	expected
auth source	environment, not source code
pagination	all pages/cursors completed
429	wait/retry policy recorded
secret in logs	none

Common beginner traps

- Hardcoding token in .py.
- Assuming page=1 is the full source.
- Immediate tight-loop retry after 429.

Now you do a similar task

Implement/fix the API TODO so it retrieves all pages with the training token. Save page count/total rows and prove the token is not tracked/logged.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What evidence would convince you that a paginated extraction is complete rather than merely successful so far?

Answer in your own words before continuing.

DE08-07 - Retries, backoff, jitter and transient failures

What you will be able to do

Decide which failures are retryable and use bounded exponential backoff with jitter rather than infinite retries.

Why this matters in real Data Engineering

Retries can recover transient failures, but reckless retries can overload services, repeat non-idempotent actions and hide deterministic bugs.

Picture it this way

If a bus is delayed, waiting and trying again can work. If you have the wrong ticket, trying the same ticket 100 times does not.

Start from zero

- Retry selected transient failures such as timeouts/connection resets, 429 and some 5xx according to API semantics.
- Do not retry permanent/deterministic conditions blindly: malformed request, invalid credentials, schema incompatibility.
- Bound attempts and total elapsed time.
- Exponential backoff increases delay; jitter adds randomness so many clients do not retry simultaneously.
- Retries are safest when the request/action is idempotent or protected by idempotency keys/state.
- Log attempt number, classification and final outcome, not secrets/full huge bodies.

Teacher demonstration

```
# Conceptual retry delay
base = 1.0
for attempt in range(max_attempts):
    delay = min(30, base * (2 ** attempt)) + random.uniform(0, 0.5)
    # sleep(delay) only for classified transient failure
```

Read the example like English

- Attempt 1 waits less than later attempts.
- Maximum delay/attempt count prevents uncontrolled waiting.
- Jitter avoids synchronized retry storms.
- The error classifier is more important than the formula itself.

Expected check

error	typical action
timeout/connection reset	bounded retry
429	wait per Retry-After/backoff
500/503	possibly retry
401	repair auth, not identical loop
schema mismatch	quarantine/repair

Common beginner traps

- while True retry loops.
- Retrying POST/side-effect operations without idempotency semantics.
- Calling every exception transient.

Now you do a similar task

Run the supplied transient retry lab or mock endpoint. Save attempt/status/timing evidence and classify one retryable and one non-retryable failure.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why is classifying the failure more important than simply adding "three retries"?

Answer in your own words before continuing.

DE08-08 - Database extraction and safe query boundaries

What you will be able to do

Extract from a relational database using a bounded SELECT, stable key/order and read-only-safe boundary.

Why this matters in real Data Engineering

Database sources can be huge and mutable. SELECT * without predicate/window can create expensive, inconsistent or unreproducible extracts.

Picture it this way

Database extraction is like photocopying records from a live office: define which drawers/date/key range are in scope and record the cutoff so another person can reproduce the batch.

Start from zero

- Connect with least required privileges; ingestion readers usually do not need schema ownership/write access.
- Select explicit columns rather than depending on changing column order.
- Use a stable incremental predicate/key and deterministic ordering when chunking.
- Use parameterized queries for values.
- For a consistent snapshot across many pages/chunks, understand transaction/isolation/source semantics.
- Record source query/window/high-water mark and extracted count.

Teacher demonstration

```
python create_source_db.py
python src/database_extract_TODO.py
# Compare the TODO with the controlled source/db created for the lab.
```

Read the example like English

- The project creates a local source database so the exercise is repeatable.
- The extraction should be bounded and record what range/window it read.
- Use parameters rather than string-building predicates.
- Land the extracted raw result before later transformation.

Expected check

proof	expected
query	explicit columns + bounded predicate
ordering	stable key/order if chunked
row count	recorded
source write	none

Common beginner traps

- SELECT * over the entire production-like source as a default.
- String-concatenating dates/IDs into SQL.
- Updating source rows from the ingestion job.

Now you do a similar task

Create the local source DB and implement the extract TODO for a bounded date/id range. Save query/window, row count and landing artifact.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

If source rows can change while you page through them, what consistency problem might occur?

Answer in your own words before continuing.

DE08-09 - Historical batch landing and raw metadata

What you will be able to do

Land historical batches with immutable raw identity and enough metadata to reproduce their origin/order.

Why this matters in real Data Engineering

Backfills often involve many old files. If you overwrite by filename or process in accidental order, history and replay become unreliable.

Picture it this way

Historical landing is like digitizing old monthly boxes: every box gets a receipt identifying source period, arrival/run and exact content before any reorganization.

Start from zero

- Keep source logical date/event period separate from ingestion processing time.
- Use deterministic naming/metadata that prevents accidental overwrite.
- Content hash can prove whether two differently named files are actually identical.
- Landing order is not automatically event-time order; sort/process according to explicit business key/window.
- Record backfill run ID and source range.
- Do not advance normal incremental state until the backfill policy says it is safe.

Teacher demonstration

```
python src/land_historical_reference.py
# Read BACKFILL_REPLAY_RULES_v1.0.txt
# Inspect landing metadata/names rather than only transformed output.
```

Read the example like English

- The reference is a teaching demonstration of historical landing mechanics.
- Raw objects should be individually addressable after the run.
- Backfill run/window metadata keeps old loads distinguishable from daily ingestion.
- Normal high-water state and backfill state need an explicit relationship.

Expected check

metadata	why
event/logical date	when source data belongs
ingested_at	when pipeline received it
run_id	which execution landed it
content hash	identity/dedup evidence

Common beginner traps

- Naming historical files only by arrival timestamp and losing source period.
- Overwriting an already-landed object.
- Moving the normal high-water mark backward/forward accidentally during backfill.

Now you do a similar task

Use two historical source files to create a landing manifest with logical date, ingestion time/run and content hash. Explain how a replay would identify the same object.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why should event/logical date and ingestion time be stored as separate concepts?

Answer in your own words before continuing.

DE08-10 - Incremental loads and high-water marks

What you will be able to do

Use a high-water mark/checkpoint to extract only new data without skipping or duplicating boundary records.

Why this matters in real Data Engineering

Incremental loads reduce repeated work, but a badly chosen watermark can silently lose late rows or double boundary rows.

Picture it this way

A high-water mark is a bookmark. The difficult part is defining whether the next read starts after the bookmark, at the bookmark with dedup, or over a safety overlap.

Start from zero

- Choose a stable monotonic source field when possible: immutable increasing ID or reliable updated_at + tie-breaker key.
- Define inclusive/exclusive boundary precisely: > last_value vs >= plus dedup.
- Composite state such as (updated_at, id) avoids ties at the same timestamp.
- Advance state only after durable landing/validation according to your atomicity policy.
- Persist checkpoint with run metadata; do not keep it only in process memory.
- Late-arriving/backdated records can defeat a pure event-time watermark; use overlap/CDC/reconciliation when source behavior requires.

Teacher demonstration

```
# Safer composite predicate idea
WHERE (updated_at, order_id) > (:last_ts, :last_id)
ORDER BY updated_at, order_id
# Save new checkpoint only AFTER the batch is safely landed.
```

Read the example like English

- The tuple predicate creates a deterministic order among equal timestamps.
- The previous checkpoint belongs to the last successful durable run, not the currently attempted run.
- If load fails after extraction, replay uses the old checkpoint so data is not skipped.

Expected check

state	rule
read start	defined inclusive/exclusive semantics
tie handling	secondary stable key
advance	after durable success
failure	old checkpoint retained

Common beginner traps

- Saving checkpoint before output is durable.
- Using only second-level timestamp when many rows share it.
- Assuming updated_at never changes backward/late.

Now you do a similar task

Implement or describe an incremental extraction over the supplied local DB using (updated_at,id) or another defensible key. Simulate a failure before state save and explain replay.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why can saving the new watermark before the landing write finishes cause permanent data loss?

Answer in your own words before continuing.

DE08-11 - CDC concepts and change-event semantics

What you will be able to do

Interpret CDC insert/update/delete events and explain how event semantics differ from querying current table state.

Why this matters in real Data Engineering

Change Data Capture provides changes, not just snapshots. A consumer must understand operation type, key, ordering and replay behavior.

Picture it this way

A snapshot tells you what a whiteboard looks like now. CDC is a camera feed of every meaningful edit: write, erase, rewrite.

Start from zero

- CDC event typically identifies table/entity key, operation type and before/after or changed values plus ordering metadata.
- An UPDATE event is not a new independent business entity; it changes state for an existing key.
- DELETE/tombstone semantics must be handled explicitly or stale rows remain downstream.
- At-least-once delivery means duplicate change events are possible; consumer needs idempotent application.
- Ordering is often guaranteed only within a partition/key stream, not globally.
- CDC position/LSN/offset is a checkpoint distinct from business event timestamps.

Teacher demonstration

```
# Simplified event examples
{"op":"c","id":"O1","after":{"status":"NEW"},"pos":101}
{"op":"u","id":"O1","after":{"status":"PAID"},"pos":102}
{"op":"d","id":"O1","before":{"status":"PAID"},"pos":103}
```

Read the example like English

- Applying all three in order to a current-state table should end with O1 absent/tombstoned according to policy.
- Reapplying event pos=102 should not create a second O1 row.
- The CDC position helps resume the stream; the business timestamp answers a different question.

Expected check

operation	consumer action
create/insert	insert new state
update	update/upsert key
delete	delete/tombstone
duplicate event	no duplicate business effect

Common beginner traps

- Treating CDC stream as append-only fact data without understanding operation semantics.
- Ignoring deletes.
- Using wall-clock processing time as CDC resume position.

Now you do a similar task

Use the supplied CDC guided event file/lab. Starting from empty state, apply create/update/delete events in order and describe the final current state plus replay behavior.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why is a CDC update event different from simply receiving another independent row?

Answer in your own words before continuing.

DE08-12 - Schema drift, compatibility, time zones and quarantine

What you will be able to do

Detect schema drift and timezone ambiguity, classify compatible/incompatible changes and quarantine unsafe input.

Why this matters in real Data Engineering

External producers change. If your parser silently accepts changed meaning, downstream data can become wrong without an obvious crash.

Picture it this way

A data contract is a plug/socket agreement. Adding an optional pin may be compatible; changing voltage/type on an existing pin can be dangerous.

Start from zero

- Schema drift includes added/removed/renamed fields, type changes and semantic changes that may not be machine-detectable.
- Define required vs optional fields and compatibility policy.
- Unknown optional fields can sometimes be preserved/ignored deliberately; missing required keys usually fail/quarantine.
- Timestamp strings without timezone/offset need an explicit source timezone contract; never guess local timezone silently.
- Normalize to an agreed instant representation (often UTC) while retaining source timezone/context if business needs it.
- Quarantine should preserve offending raw input, reason, observed schema and run metadata.

Teacher demonstration

```
# Contract idea
required = {"order_id": "string", "updated_at": "timestamp-with-offset"}
optional = {"coupon_code": "string"}
# unsafe examples:
# order_id disappears
# amount changes from numeric to arbitrary text
# timestamp loses offset/time-zone contract
```

Read the example like English

- Compatibility is a policy decision, not simply "JSON parsed".
- Timezone normalization must be based on known source semantics.
- Quarantine is not deletion; it is an evidence/repair route.

Expected check

change	possible policy
new optional field	compatible after review
missing required ID	quarantine/fail
type/meaning change	incompatible until contract update
timestamp without defined zone	unsafe

Common beginner traps

- Auto-adding every new field to downstream schema.
- Interpreting naive timestamps using the laptop timezone.
- Dropping quarantined raw data/reason.

Now you do a similar task

Run the drift/time-zone guided lab. Produce a compatibility decision table for three changed records and save quarantined raw + reason for the unsafe one.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why is a timestamp like 2026-08-01 10:00 incomplete without a timezone contract?

Answer in your own words before continuing.

DE08-13 - Checkpoint/state management and replay safety

What you will be able to do

Design persistent checkpoint state so a failed/replayed ingestion neither skips work nor falsely marks incomplete work successful.

Why this matters in real Data Engineering

Process memory disappears, jobs crash and retries happen. State is part of the pipeline correctness model.

Picture it this way

Checkpoint state is a signed receipt saying exactly how far a successful run got. You update the receipt only after the corresponding package is safely stored.

Start from zero

- State can be a watermark, cursor, CDC offset, processed content identity or combination.
- Persist state durably outside transient process memory.
- Associate state update with run/output success; when atomic transaction across systems is impossible, define ordering and recovery steps.
- Store previous/current candidate state and run status so recovery can distinguish attempted vs committed progress.
- Replay from last committed checkpoint; outputs must be idempotent/content-addressed or deduplicated.
- Backfills may need separate state namespace so they do not corrupt normal incremental progress.

Teacher demonstration

```
# Simple state document idea
{
  "source": "orders_api",
  "committed_cursor": "abc123",
  "run_id": "run-20260828-001",
  "status": "SUCCESS"
}
```

Read the example like English

- The committed cursor represents work proven durable.
- A running/failed candidate cursor should not replace it prematurely.
- The run_id connects state to raw/output/log evidence.

Expected check

failure point	safe restart
before landing	same committed checkpoint
after landing before state commit	replay + idempotent landing
after state commit	start after committed state

Common beginner traps

- Keeping checkpoint only in a local variable.
- Updating state before durable output.
- Sharing one unstructured state file among unrelated sources/backfills.

Now you do a similar task

Design state JSON for one source and simulate three failure points. Explain exactly which checkpoint the next run reads and whether landed output may be repeated.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What makes a checkpoint "committed" rather than merely observed during a running job?

Answer in your own words before continuing.

DE08-14 - Idempotent landing and content identity

What you will be able to do

Make landing idempotent by identifying content so the same source object/request replay does not create uncontrolled duplicate raw copies/effects.

Why this matters in real Data Engineering

Retries and backfills frequently present the same content again. Idempotency means repeated execution reaches the same durable result, not "never retry".

Picture it this way

If the same parcel is scanned twice, its tracking/content identity should tell the warehouse it is the same parcel rather than two separate orders.

Start from zero

- Content identity can use source object ID/version/ETag or a cryptographic hash when appropriate.
- Idempotent landing policy might reuse an existing object, mark duplicate receipt, or write a run manifest pointing to the same immutable content.
- Filename alone is weak identity because names can change or be reused.
- Hash alone does not capture business semantics/version history if identical content at different source times matters; combine metadata deliberately.
- Downstream loads also need key/upsert/dedup semantics; idempotent raw landing alone is not enough.
- Record what happened on a duplicate replay rather than silently skipping with no audit evidence.

Teacher demonstration

```
import hashlib
def sha256_bytes(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()
# Use the content hash in a landing manifest/object identity policy.
```

Read the example like English

- Same byte content produces the same SHA-256 digest.
- A changed byte produces a different digest with overwhelming probability.
- The digest helps prove content identity; it does not tell you whether two business records are semantically duplicates.

Expected check

identity	strength
filename only	weak/reusable
source version/etag	strong if source contract stable
content hash	strong byte identity
business key	semantic record identity

Common beginner traps

- Calling hash a business primary key automatically.
- Writing duplicate raw files endlessly on every retry without a policy.
- Skipping a duplicate and not recording that the replay occurred.

Now you do a similar task

Run the content-identity guided task on one source file twice. Save digest/landing behavior and explain how the second receipt is represented without duplicating business effect.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What is the difference between byte/content identity and business-record identity?

Answer in your own words before continuing.

DE08-15 - Late-arriving data, event time and processing time

What you will be able to do

Distinguish event time from processing/ingestion time and design a policy for late-arriving data.

Why this matters in real Data Engineering

Data can arrive minutes or days after the event occurred. Watermarks, partitions and reports are wrong if arrival time is mistaken for business event time.

Picture it this way

An order may be placed Monday but the courier delivers the file Wednesday. Monday is event time; Wednesday is processing/arrival time.

Start from zero

- **Event time** describes when the business event happened; **processing/ingestion time** describes when your pipeline handled it.
- Late data can arrive after a daily partition/report was already produced.
- Partitioning by event date often matches analytics, but late data means old partitions may need safe update/reprocessing.
- Incremental state based only on event time can miss late records if the source does not update a monotonic ingestion/change field.
- Define lateness SLA/watermark/reopen policy: e.g. reprocess last N days or use CDC/source update timestamp.
- Record both timestamps so latency and late-arrival behavior are measurable.

Teacher demonstration

```
# Example record
{
  "order_id": "O9",
  "event_time": "2026-08-20T10:00:00Z",
  "ingested_at": "2026-08-23T06:30:00Z"
}
# ingestion_lag = ingested_at - event_time
```

Read the example like English

- The record belongs to Aug 20 business/event date even though pipeline saw it Aug 23.
- A lateness policy determines whether Aug 20 outputs are reopened/backfilled.
- Lag becomes an observability metric.

Expected check

time	question answered
event_time	when did business event occur?
ingested_at	when did we receive/process it?
lag	how late was it?

Common beginner traps

- Partitioning only by ingestion date then reporting it as business date.
- A watermark that permanently closes event dates too early.
- Silently rewriting old aggregates without run/backfill evidence.

Now you do a similar task

Create three records: on-time, 1-day late, 5-day late. Classify them under a stated lateness policy and explain which partitions/state need reprocessing.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

Why can a pure event-time high-water mark miss late-arriving records?

Answer in your own words before continuing.

DE08-16 - Ingestion observability: counts, latency and run metadata

What you will be able to do

Record ingestion observability that lets you answer what ran, how much data moved, how long it took and where rows were lost/rejected.

Why this matters in real Data Engineering

Without run metadata, an empty target could mean no source data, failed extraction, filter bug or rejected batch. Observability reduces guesswork.

Picture it this way

Observability is the shipment tracking screen: run ID, source, start/end, counts, latency, status and exception/rejection details.

Start from zero

- Give every run a unique run_id and source/window identity.
- Record start/end/duration, page/file counts, extracted/landed/staged/rejected counts and checkpoint movement.
- Reconcile counts at boundaries instead of only logging "success".
- Latency metrics can include source-to-ingestion lag and run duration.
- Status should distinguish SUCCESS, FAILED and INCOMPLETE/PARTIAL as needed.
- Logs/metrics must not contain secrets; retain enough key samples/reasons for diagnosis without dumping sensitive payloads.

Teacher demonstration

```
run_summary = {
  "run_id": run_id,
  "source": "orders_api",
  "status": "SUCCESS",
  "extracted": 120,
  "staged": 117,
  "rejected": 3,
  "pages": 4,
  "checkpoint_before": "c17",
  "checkpoint_after": "c21"
}
assert run_summary["extracted"] == run_summary["staged"] + run_summary["rejected"]
```

Read the example like English

- The assertion is a reconciliation, not merely a log.
- Checkpoint before/after shows exactly what progress the successful run committed.
- Run ID ties the summary to raw artifacts and detailed logs.

Expected check

metric	why
extracted/landed	source completeness
staged/rejected	quality accounting
duration	runtime regression

metric	why
ingestion lag	source lateness
checkpoint movement	incremental progress

Common beginner traps

- Only logging stack traces after failure.
- No row/page counts.
- Logging tokens/passwords/payloads indiscriminately.

Now you do a similar task

Add a run summary to one ingestion exercise. Include run ID, window, page/file counts, row reconciliation, duration and checkpoint before/after.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

If a job says SUCCESS but extracted=100, staged=90 and rejected=0, what should your first suspicion be?

Answer in your own words before continuing.

DE08-17 - Ingestion integration, reconciliation and backfill proof

What you will be able to do

Combine file/API/database ingestion into a defensible run with raw preservation, state, idempotency, observability and backfill/replay proof.

Why this matters in real Data Engineering

Production ingestion fails at boundaries, not in isolated syntax questions. The integration lesson makes you reason about the entire lifecycle.

Picture it this way

You are running a receiving department: every source shipment needs identity, a safe dock, inspection, exception handling, progress state and an auditable receipt.

Start from zero

- Map each source to its contract, raw landing identity and staging shape.
- Do not advance state for an incomplete extraction.
- Reconciliation must explain every source object/row/page as accepted, rejected/quarantined or explicitly excluded.
- Replay the exact same input and prove raw/business effects do not multiply unexpectedly.
- Backfill an older window without corrupting normal incremental checkpoint.
- Use run metadata to prove completeness and latency.
- The final handoff should state failure policy and how to resume safely after each major failure point.

Teacher demonstration

```
# Integration proof checklist
source_count_or_pages == landed_count_or_pages
parsed_rows == staged_rows + rejected_rows
checkpoint_after only exists for SUCCESS
replay_same_input -> no unintended duplicate durable rows
backfill_state does not overwrite normal incremental state
```

Read the example like English

- Each line is a correctness invariant you can test.
- The exact counts depend on the supplied project scenario; save observed values.
- A clean second run is stronger evidence than a one-time successful run.

Expected check

dimension	proof
completeness	all pages/files/windows accounted
quality	valid + rejected reconciliation
state	checkpoint moves after success only
idempotency	same input replay safe
backfill	historical run isolated/audited

Common beginner traps

- Calling integration complete because each individual script ran once.
- Advancing state on partial success.
- Backfill by manually editing checkpoint with no audit trail.

Now you do a similar task

Complete the module practical using the supplied stage08 project. Then replay the same input and perform one historical/backfill run. Save the run manifests and explain every count/state decision.

How to prove it

- Save actual code/config/commands used.
- Save at least one independent output/state/reconciliation check.
- State the important grain/window/state/identity assumption.
- If result differs, diagnose before move-on.

STOP - check your understanding

What three pieces of evidence would you demand before calling an ingestion run safe to replay?

Answer in your own words before continuing.